UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623

Database Programming

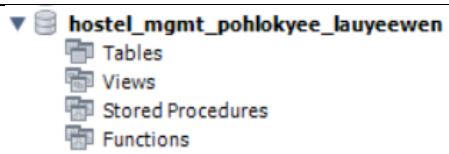
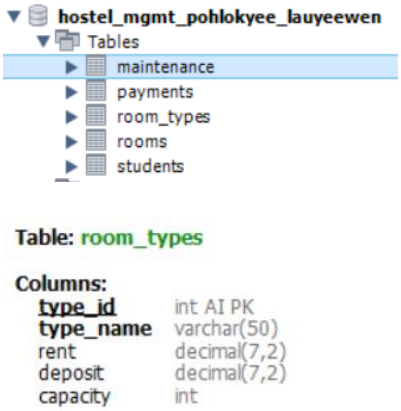
2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	Lau Yee Wen
Matric Id	:	A23CS0099
Section	:	Section 01

Name	:	Poh Lok Yee
Matric Id	:	A23CS0262
Section	:	Section 01

Task 1: Database Creation and Integrity Constraints (CLO1)

SQL Command	Output / Result
1. Create new database	
`` CREATE DATABASE hostel_mgmt_PohLokYee_LauYeeWen; USE hostel_mgmt_PohLokYee_LauYeeWen; ``` CREATE DATABASE are used to create new database	
2. Create table and apply contraints	
`` -- 2&3&4. Create tables, apply aonstriants CREATE TABLE IF NOT EXISTS room_types(type_id INT PRIMARY KEY AUTO_INCREMENT, type_name VARCHAR (50) NOT NULL UNIQUE, rent DECIMAL (7,2) NOT NULL, deposit DECIMAL (7,2) NOT NULL, capacity INT NOT NULL); ``` Create table **'room_types'** using CREATE TABLE, set type_id as the PRIMARY KEY, and apply NOT NULL and UNIQUE constraints to other attributes.	
`` CREATE TABLE IF NOT EXISTS rooms(room_id INT PRIMARY KEY AUTO_INCREMENT, type_id INT NOT NULL, room_no VARCHAR (10) NOT NULL UNIQUE, floor_no INT NOT NULL, is_occupied BOOLEAN DEFAULT FALSE, FOREIGN KEY (type_id) REFERENCES room_types(type_id)); ``` Create table **'rooms'** using CREATE TABLE, set room_id as the PRIMARY KEY, and define type_id as a FOREIGN KEY that references the room_types table.	
`` CREATE TABLE IF NOT EXISTS students(student_id INT PRIMARY KEY AUTO_INCREMENT, room_id INT, fname VARCHAR(50) NOT NULL, lname VARCHAR(50) NOT NULL, status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL DEFAULT 'NON_ACTIVE', checkin_date DATE, FOREIGN KEY (room_id) REFERENCES rooms(room_id)); ``` Create table **'students'** using CREATE TABLE, set student_id as the PRIMARY KEY, and define room_id as a FOREIGN KEY that references the rooms table.	

```sql
CREATE TABLE IF NOT EXISTS maintenance(
    maint_id INT PRIMARY KEY AUTO_INCREMENT,
    room_id INT,
    issue_desc VARCHAR(255) NOT NULL,
    severity ENUM('LOW','MEDIUM','HIGH') NOT NULL,
    status ENUM('OPEN','RESOLVED') NOT NULL DEFAULT 'OPEN',
    reported_on DATE NOT NULL,
    resolved_on DATE ,
    FOREIGN KEY(room_id) REFERENCES rooms(room_id)
);
```

Create table **'maintenance'** using CREATE TABLE, set maint_id as the PRIMARY KEY, and define room_id as a FOREIGN KEY that references the rooms table.

Table: maintenance

Columns:

maint_id	int AI PK
room_id	int
issue_desc	varchar(255)
severity	enum('LOW','MEDIUM','HIGH')
status	enum('OPEN','RESOLVED')
reported_on	date
resolved_on	date

```sql
CREATE TABLE IF NOT EXISTS payments(
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    amount DECIMAL(6,2) NOT NULL,
    paid_on DATE NOT NULL,
    method ENUM('CASH','FPX','CARD','TNG') NOT NULL,
    note VARCHAR(255),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

Create table **'payments'** using CREATE TABLE, set payment_id as the PRIMARY KEY, and define student_id as a FOREIGN KEY that references the students table.

Table: payments

Columns:

payment_id	int AI PK
student_id	int
amount	decimal(6,2)
paid_on	date
method	enum('CASH','FPX','CARD','TNG')
note	varchar(255)

3. Demonstrate schema maintenance

```sql
ALTER TABLE students
ADD email VARCHAR(100) UNIQUE;
```

Added a unique email field to students table.

```sql
CREATE TABLE temp_test (id INT);
DROP TABLE temp_test;
```

Created and dropped a temporary table.

Table: students

Columns:

student_id	int(11) AI PK
room_id	int(11)
fname	varchar(50)
lname	varchar(50)
status	enum('ACTIVE','NON_ACTIVE')
checkin_date	date
email	varchar(100)

Email field is added to the students table.

Queries execute successfully.

Task 2 – Data Manipulation and Filtering (CLO2)

SQL Command	Output / Result

1. Insert at least 10 rows per table with realistic and varied data.

```sql
-- Task 2 - Data Manipulation and Filtering (CLO2)
-- 1. Data Insertion (DML) with at least 10 rows realistic and varied data per table

-- room_types table
INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES
    ('Single Standard', 450.00, 200.00, 1),
    ('Single Plus', 550.00, 250.00, 1),
    ('Single Premium', 650.00, 350.00, 1),
    ('Double Standard', 750.00, 400.00, 2),
    ('Double Plus', 800.00, 400.00, 2),
    ('Double Premium', 850.00, 450.00, 2),
    ('Premium Loft', 1000.00, 500.00, 2),
    ('Premium Suite', 1300.00, 650.00, 2),
    ('Family Standard', 1600.00, 800.00, 4),
    ('Family Deluxe', 2000.00, 1200.00, 4);
```

Inserted 10 room type records to **'room_types'** table using INSERT INTO… VALUES

Result Grid | Filter Rows: | Export:

type_id	type_name	rent	deposit	capacity
1	Single Standard	450.00	200.00	1
2	Single Plus	550.00	250.00	1
3	Single Premium	650.00	350.00	1
4	Double Standard	750.00	400.00	2
5	Double Plus	800.00	400.00	2
6	Double Premium	850.00	450.00	2
7	Premium Loft	1000.00	500.00	2
8	Premium Suite	1300.00	650.00	2
9	Family Standard	1600.00	800.00	4
10	Family Deluxe	2000.00	1200.00	4

```sql
-- rooms table
INSERT INTO rooms (type_id, room_no, floor_no, is_occupied) VALUES
    (1, 'A101', 1, FALSE),
    (2, 'A102', 1, FALSE),
    (3, 'A203', 2, FALSE),
    (4, 'B201', 2, FALSE),
    (5, 'B202', 2, FALSE),
    (6, 'B305', 3, FALSE),
    (7, 'C108', 1, FALSE),
    (8, 'C118', 1, FALSE),
    (9, 'C405', 4, FALSE),
```

Inserted 10 room records to **'rooms'** table using INSERT INTO… VALUES

room_id	type_id	room_no	floor_no	is_occupied
1	1	A101	1	1
2	2	A102	1	1
3	3	A203	2	0
4	4	B201	2	1
5	5	B202	2	1
6	6	B305	3	0
7	7	C108	1	1
8	8	C218	2	1
9	9	C405	4	1
10	10	C202	2	0

```sql
-- student table
INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES
    (1, 'Adib', 'Zikri', 'ACTIVE', '2025-01-01', 'adib@graduate.utm.my'),
    (2, 'Yee Wen', 'Lau', 'ACTIVE', '2025-01-03', 'lauyeewen@graduate.utm.my'),
    (NULL, 'Lok Yee', 'Poh', 'NON_ACTIVE', '2025-02-03', 'pohlokyee@graduate.utm.my'),
    (4, 'Yasmin', 'Batrisya', 'ACTIVE', '2025-02-01', 'yasmin@graduate.utm.my'),
    (5, 'Cheryl', 'Cheong', 'ACTIVE', '2025-02-14', 'cheryl@graduate.utm.my'),
    (NULL, 'Aqmal', 'Azizi', 'NON_ACTIVE', '2025-03-05', 'aqmal@graduate.utm.my'),
    (7, 'Brendan', 'Ng', 'ACTIVE', '2025-01-02', 'brendan@graduate.utm.my'),
    (8, 'Sabrina', 'Heng', 'ACTIVE', '2025-01-01', 'sabrina@graduate.utm.my'),
    (9, 'Jane', 'Woo', 'ACTIVE', '2025-01-04', 'jane@graduate.utm.my'),
    (NULL, 'Jia Jia', 'Lee', 'NON_ACTIVE', '2025-01-05', 'leejiajia@graduate.utm.my');
```

Insert 10 student records to **'students'** table using INSERT INTO… VALUES

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Adib	Zikri	ACTIVE	2025-01-01	adib@graduate.utm.my
2	2	Yee Wen	Lau	ACTIVE	2025-01-03	lauyeewen@graduate.utm.my
3	NULL	Lok Yee	Poh	NON_ACTIVE	2025-02-03	pohlokyee@graduate.utm.my
4	4	Yasmin	Batrisya	ACTIVE	2025-02-01	yasmin@graduate.utm.my
5	5	Cheryl	Cheong	ACTIVE	2025-02-14	cheryl@graduate.utm.my
6	NULL	Aqmal	Azizi	NON_ACTIVE	2025-03-05	aqmal@graduate.utm.my
7	7	Brendan	Ng	ACTIVE	2025-01-02	brendan@graduate.utm.my
8	8	Sabrina	Heng	ACTIVE	2025-01-01	sabrina@graduate.utm.my
9	9	Jane	Woo	ACTIVE	2025-01-04	jane@graduate.utm.my
10	NULL	Jia Jia	Lee	NON_ACTIVE	2025-01-05	leejiajia@graduate.utm.my

```sql
-- maintenance table
INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
    (1, 'Broken desk', 'MEDIUM', 'RESOLVED', '2025-06-10', '2025-06-15'),
    (2, 'Water leak in bathroom', 'HIGH', 'RESOLVED', '2025-06-19', '2025-06-20'),
    (3, 'Aircond not cold', 'LOW', 'RESOLVED', '2025-07-04', '2025-07-10'),
    (4, 'Door lock issue', 'MEDIUM', 'RESOLVED', '2025-07-08', '2025-07-10'),
    (5, 'Noisy neighbors', 'LOW', 'RESOLVED', '2025-08-21', '2025-08-28'),
    (6, 'Fan not working', 'MEDIUM', 'RESOLVED', '2025-09-23','2025-09-26' ),
    (7, 'Leaking pipe', 'MEDIUM', 'OPEN', '2025-10-21', NULL),
    (8, 'Broken light bulb', 'LOW', 'OPEN', '2025-10-21', NULL),
    (9, 'Unstable WiFi connection', 'LOW', 'OPEN', '2025-10-24', NULL),
    (10, 'Bathroom door jammed', 'HIGH', 'OPEN', '2025-11-01', NULL);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	Broken desk	MEDIUM	RESOLVED	2025-06-10	2025-06-15
2	2	Water leak in bathroom	HIGH	RESOLVED	2025-06-19	2025-06-20
3	3	Aircond not cold	LOW	RESOLVED	2025-07-04	2025-07-10
4	4	Door lock issue	MEDIUM	RESOLVED	2025-07-08	2025-07-10
5	5	Noisy neighbors	LOW	RESOLVED	2025-08-21	2025-08-28
6	6	Fan not working	MEDIUM	RESOLVED	2025-09-23	2025-09-26
7	7	Leaking pipe	MEDIUM	OPEN	2025-10-21	NULL
8	8	Broken light bulb	LOW	OPEN	2025-10-21	NULL
9	9	Unstable WiFi connection	LOW	OPEN	2025-10-24	NULL
10	10	Bathroom door jammed	HIGH	OPEN	2025-11-01	NULL

Insert 10 maintenance records to **'maintenance'** table using INSERT INTO… VALUES

```
-- payment table
INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES
    (1, 450.00, '2025-01-01', 'CASH', 'January Rent'),
    (2, 550.00, '2025-01-05', 'FPX', 'January Rent'),
    (3, 650.00, '2025-02-01', 'CARD', 'February Rent'),
    (4, 750.00, '2025-02-05', 'TNG', 'February Rent'),
    (5, 800.00, '2025-01-10', 'CASH', 'January Rent'),
    (6, 850.00, '2025-03-15', 'FPX', 'March Rent'),
    (7, 1000.00, '2025-01-20', 'CARD', 'January Rent'),
    (8, 1300.00, '2025-02-01', 'TNG', 'February Rent'),
    (9, 1600.00, '2025-01-25', 'CASH', 'January Rent'),
    (10, 2000.00, '2025-02-10', 'FPX', 'February Rent');
```

Insert 10 payment records to **'payments'** table using INSERT INTO… VALUES

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_id	student_id	amount	paid_on	method	note
1	1	450.00	2025-01-01	CASH	January Rent
2	2	550.00	2025-01-05	FPX	January Rent
3	3	650.00	2025-02-01	CARD	February Rent
4	4	750.00	2025-02-05	TNG	February Rent
5	5	800.00	2025-01-10	CASH	January Rent
6	6	850.00	2025-03-15	FPX	March Rent
7	7	1000.00	2025-01-20	CARD	January Rent
8	8	1300.00	2025-02-01	TNG	February Rent
9	9	1600.00	2025-01-25	CASH	January Rent
10	10	2000.00	2025-02-10	FPX	February Rent

2. Update rooms.is_occupied to TRUE if there is a student in that room who is ACTIVE

```
-- 2. UPDATE and DELETE Operations
-- UPDATE OPERATIONS
UPDATE rooms
INNER JOIN students ON rooms.room_id = students.room_id
SET rooms.is_occupied = TRUE
WHERE students.status ='ACTIVE';
```

It links the rooms table and the students table by using INNER JOIN. Then it checks the joined result where if the student's status is ACTIVE, it UPDATE that room's is occupied column in rooms table to TRUE.

room_id	type_id	room_no	floor_no	is_occupied
1	1	A101	1	1
2	2	A102	1	1
3	3	A203	2	0
4	4	B201	2	1
5	5	B202	2	1
6	6	B305	3	0
7	7	C108	1	1
8	8	C218	2	1
9	9	C405	4	1
10	10	C202	2	0

Some of the is_occupied column changed from 0 (False) to 1 (True).

Delete maintenance records where status = 'RESOLVED' and the reported date is older than 60 days

```
-- DELETE OPERATION
DELETE FROM maintenance
WHERE status ='RESOLVED'
    AND reported_on < CURDATE() - INTERVAL 60 DAY;
```

It DELETE records from the maintenance table WHERE the issue status is 'RESOLVED' and the reported_on date is older than 60 days.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

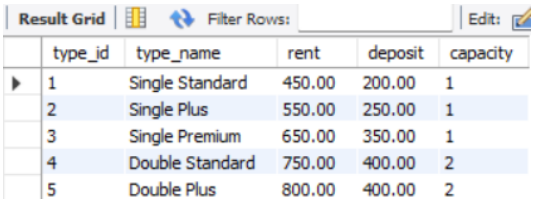
maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
6	6	Fan not working	MEDIUM	RESOLVED	2025-09-23	2025-09-26
7	7	Leaking pipe	MEDIUM	OPEN	2025-10-21	NULL
8	8	Broken light bulb	LOW	OPEN	2025-10-21	NULL
9	9	Unstable WiFi connection	LOW	OPEN	2025-10-24	NULL
10	10	Bathroom door jammed	HIGH	OPEN	2025-11-01	NULL

The record before 60 days ago are deleted successfully.

3. Display room types by using BETWEEN where rent is between RM400 and RM800.

```
-- 3. Data Retrieval and Filtering Queries:
-- Queries 1
SELECT *
FROM room_types
WHERE rent BETWEEN 400 AND 800;
```

Filtering Query using BETWEEN.



type_id	type_name	rent	deposit	capacity
1	Single Standard	450.00	200.00	1
2	Single Plus	550.00	250.00	1
3	Single Premium	650.00	350.00	1
4	Double Standard	750.00	400.00	2
5	Double Plus	800.00	400.00	2

Retrieve students whose first names start with the letter 'A' by using LIKE.

```
-- Queries 2
SELECT *
FROM students
WHERE fname LIKE 'A%';
```

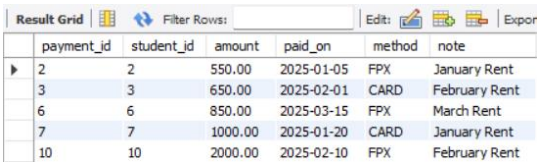
Filtering Query using LIKE to find out name starting with A.

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Adib	Zikri	ACTIVE	2025-01-01	adib@graduate.utm.my
6	NULL	Aqmal	Azizi	NON_ACTIVE	2025-03-05	aqmal@graduate.utm.my

Display the payments where the payment method is IN 'FPX' and 'CARD'.

```
-- Queries 3
SELECT *
FROM payments
WHERE method IN ('FPX' ,'CARD');
```

Filtering Query using IN.

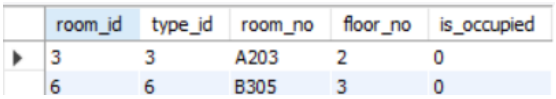


payment_id	student_id	amount	paid_on	method	note
2	2	550.00	2025-01-05	FPX	January Rent
3	3	650.00	2025-02-01	CARD	February Rent
6	6	850.00	2025-03-15	FPX	March Rent
7	7	1000.00	2025-01-20	CARD	January Rent
10	10	2000.00	2025-02-10	FPX	February Rent

Combine AND, OR, and NOT in one logical filter query.

```
-- Queries 4
SELECT *
FROM rooms
WHERE (floor_no=2 OR floor_no=3) AND NOT is_occupied;
```

The filtering Query using OR, AND and NOT to select the rooms from **'rooms'** table that the second and third block are not occupied by any student.



room_id	type_id	room_no	floor_no	is_occupied
3	3	A203	2	0
6	6	B305	3	0

4. Aggregate Function: Use function sum() to calculate the total payments collected

```
SELECT sum(amount) AS total_payments_collected
FROM payments;
```

Result Grid | Filter Row

total_payments_collected
9950.00

String Functions: Use UPPER() and CONCAT() to combine the last name and first name of the student, and make the name all uppercase.

```
SELECT UPPER (CONCAT (fname, ' ', lname)) AS full_name
FROM students;
```

Result Grid

full_name
ADIB ZIKRI
YEE WEN LAU
LOK YEE POH
YASMIN BATRISYA
CHERYL CHEONG
AQMAL AZIZI
BRENDAN NG
SABRINA HENG
JANE WOO
JIA JIA LEE

Task 3 – Reporting and Aggregation (CLO2)

SQL Command	Output / Result

1. Using CREATE VIEW to make a new virtual table which first selects necessary room properties and link the rooms table with the room_types table by using INNER JOIN. To generate variable such as n_occupants,pending_issues and is_vacant, COUNT() is used to find the total number after filtered with specific condition by using AND. The is_vacant status is found by comparing the active student count to zero, meaning the columns returns TRUE only when the room has no active occupants.

```sql
-- Task 3 - Reporting and Aggregation (CLO2)
-- 1.Create view v_room_status
    CREATE VIEW v_room_status AS
    SELECT
        r.room_no,
        rt.type_name,
        rt.rent,
        r.floor_no,
        rt.capacity,

        (SELECT COUNT(s.student_id)
        FROM students s
        WHERE s.room_id = r.room_id AND s.status = 'ACTIVE'
        ) AS n_occupants,

        (SELECT COUNT(m.maint_id)
        FROM maintenance m
        WHERE m.room_id = r.room_id AND m.status = 'OPEN'
        ) AS pending_issues,

        ((SELECT COUNT(s.student_id)
        FROM students s
        WHERE s.room_id = r.room_id AND s.status = 'ACTIVE'
        ) = 0
        ) AS is_vacant

    FROM rooms r
    INNER JOIN room_types rt ON r.type_id = rt.type_id;

    SELECT * FROM v_room_status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents:

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Single Standard	450.00	1	1	1	0	0
A102	Single Plus	550.00	1	1	1	0	0
A203	Single Premium	650.00	2	1	0	0	1
B201	Double Standard	750.00	2	2	1	0	0
B202	Double Plus	800.00	2	2	1	0	0
B305	Double Premium	850.00	3	2	0	0	1
C108	Premium Loft	1000.00	1	2	1	1	0
C118	Premium Suite	1300.00	1	2	1	1	0
C405	Family Standard	1600.00	4	4	1	1	0
C102	Family Deluxe	2000.00	1	4	0	1	1

2. Uses JOIN to link the students, rooms, and room_types tables together. Then filters with WHERE to find the only ‘ACTIVE’ students and uses GROUP BY to COUNT how many active students belong to each room type.

```sql
-- 2. Summary Queries
-- Total number of students per room type
SELECT rt.type_name, COUNT(s.student_id) AS number_of_students
FROM students s

JOIN rooms r
ON s.room_id = r.room_id

JOIN room_types rt
ON r.type_id = rt.type_id

WHERE s.status = 'ACTIVE'
GROUP BY rt.type_name;
```

Result Grid | Filter Rows:

type_name	number_of_students
Single Standard	1
Single Plus	1
Double Standard	1
Double Plus	1
Premium Loft	1
Premium Suite	1
Family Standard	1

Uses ROUND with AVG to calculate the average rent and ROUND with SUM to get the total deposit for each type.

```sql
-- Average rent and total deposit per room type
SELECT
    type_name,
    ROUND(AVG(rent), 2) AS average_rent,
    ROUND(SUM(deposit),2) AS total_deposit

FROM room_types
GROUP BY type_name;
```

Result Grid | Filter Rows:

type_name	average_rent	total_deposit
Double Plus	800.00	400.00
Double Premium	850.00	450.00
Double Standard	750.00	400.00
Family Deluxe	2000.00	1200.00
Family Standard	1600.00	800.00
Premium Loft	1000.00	500.00
Premium Suite	1300.00	650.00
Single Plus	550.00	250.00
Single Premium	650.00	350.00
Single Standard	450.00	200.00

Groups all payments by both the YEAR and MONTH they were paid, then uses SUM to calculate the total revenue collected for each specific period and sorts the results.

```sql
-- Monthly payment totals grouped by year & month
SELECT  year(paid_on) AS payment_year,
        month (paid_on) AS payment_month,
        sum(amount) AS total_monthly_payment

FROM payments
GROUP BY year(paid_on), month(paid_on)
ORDER BY payment_year, payment_month;
```

Result Grid | Filter Rows: | Export: | W

payment_year	payment_month	total_monthly_payment
2025	1	4400.00
2025	2	4700.00
2025	3	850.00

Uses JOIN to link the maintenance and rooms tables, filters with WHERE to find only 'OPEN' issues, then uses GROUP BY to count issues per floor and HAVING to show only floors with more than 2 open issues.

```sql
-- Count of OPEN maintenance issues per floor using HAVING COUNT
SELECT
    r.floor_no,
    COUNT(m.maint_id) AS open_issues

FROM maintenance m
JOIN rooms r
ON m.room_id = r.room_id
WHERE m.status = 'OPEN'
GROUP BY r.floor_no
HAVING
    COUNT(m.maint_id) > 2;
```

Result Grid | Filter Rows

floor_no	open_issues
1	3

3. This code mainly filters the active student and links them to their room, room type and payment record using INNER JOIN. UPPER() and CONCAT() are used to combined and uppercase the student names. ROUND() are used to rounding the rental fee to the nearest whole number. CASE statement categorizes rent into LOW (<600), MEDIUM (<100) and HIGH (>=1000) categories**.** COUNT() is used to find the total number after filtered with specific condition by using AND. ORDER BY and ASC are used to present the report in ascending order.

```sql
-- 3. Create final Report
SELECT
    UPPER (CONCAT (s.fname,' ', s.lname)) AS 'Student Full Name',          -- Using UPPER() and CONCAT()
    s.email AS 'Student Email',
    r.room_no AS 'Room No',
    r.floor_no AS 'Floor',
    rt.type_name AS 'Room Type',
    ROUND(rt.rent, 0) AS 'Monthly Rent',            -- Using ROUND()

    CASE
        WHEN rt.rent < 600.00 THEN 'LOW'            -- Using CASE
        WHEN rt.rent < 1000.00 THEN 'MEDIUM'        -- Rent below RM600 = LOW, RM600-RM1000 = MEDIUM
        ELSE 'HIGH'                                 -- higher than RM1000 = HIGH
    END AS 'Rent Category',

    (SELECT COUNT(m.maint_id)
     FROM maintenance m
     WHERE m.room_id = r.room_id AND m.status = 'OPEN'
    ) AS 'Pending Issues',

s.checkin_date AS 'Check In Date',
p.paid_on AS 'Rent Paid Date'

FROM students s
JOIN payments p ON  p.student_id = s.student_id
JOIN rooms r ON s.room_id = r.room_id
JOIN room_types rt ON r.type_id = rt.type_id
WHERE s.status = 'ACTIVE'
ORDER BY s.fname ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Student Full Name	Student Email	Room No	Floor	Room Type	Monthly Rent	Rent Category	Pending Issues	Check In Date	Rent Paid Date
ADIB ZIKRI	adib@graduate.utm.my	A101	1	Single Standard	450	LOW	0	2025-01-01	2025-01-01
BRENDAN NG	brendan@graduate.utm.my	C108	1	Premium Loft	1000	HIGH	1	2025-01-02	2025-01-20
CHERYL CHEONG	cheryl@graduate.utm.my	B202	2	Double Plus	800	MEDIUM	0	2025-01-09	2025-01-10
JANE WOO	jane@graduate.utm.my	C405	4	Family Standard	1600	HIGH	1	2025-01-04	2025-01-25
SABRINA HENG	sabrina@graduate.utm.my	C118	1	Premium Suite	1300	HIGH	1	2025-01-01	2025-02-01
YASMIN BATRISYA	yasmin@graduate.utm.my	B201	2	Double Standard	750	MEDIUM	0	2025-02-01	2025-02-05
YEE WEN LAU	lauyeewen@graduate.utm.my	A102	1	Single Plus	550	LOW	0	2025-01-03	2025-01-05

Table Created

Table Name	Number of Records
room_types	10
rooms	10
students	10
maintenance	10
payments	10

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (`PK, FK, NOT NULL, UNIQUE`). Logical order and referential integrity preserved.	3	
	`ALTER TABLE` (add email `UNIQUE`) and `DROP TABLE` demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data `UPDATE/DELETE` executed correctly with logical results.	1	
	Accurate use of `WHERE, BETWEEN, LIKE, IN, AND/OR/NOT` queries.	3	
	Correct use of aggregate and string functions (`COUNT, CONCAT, and more).`	2	
Task 3 Reporting & Aggregation	`VIEW` creation	2	
	Use of `GROUP BY, HAVING, ROUND, and CASE` with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
		Total (20)	